

Soda Can Heat Transfer Experiment

ME 30902

Oscar Ivy, oaivy@wpi.edu

12/5/24

Objective:

The goal for this experiment is to accurately record temperature using a thermistor, create a soda can that has a 20% higher heat transfer coefficient (K), using the thermistor to test the heat changes.

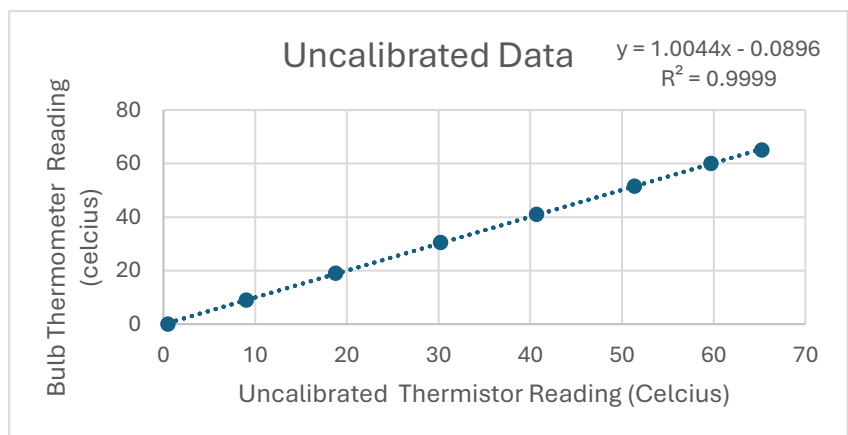
Design:

Temperature Data:

The thermistor was chosen for this experiment primarily because of the reference needed for the thermocouple setup. Should a reference come out of the ice bath or the temperature change, the reference would get messed up. Also, in testing both systems, the thermistor was more accurate, while the thermocouple could record temperature change much faster; since the temperature of the soda change likely wont change radically over a few minutes, the thermistor is the best for this experiment.

The thermistor's resistance (which is proportional to temperature) is calculated in Micro-Python code after collecting voltage information from a voltage divider circuit using an ADC chip. After temperature data was recorded, the thermistor was calibrated using a Linear Regression equation.

Thermistor Temperature (Celsius)	Thermometer Temperature (Celsius)
0.491	0
9.033	9
18.78	19
30.22	30.5
40.67	41
51.37	51.5
59.7	60
65.23	65



The thermistor was calibrated using the regression equation above in the temperature readings.

Soda Cans:

To test the effect of the design changes to the original soda can, controlling all other variables besides those intentionally changed is essential. To do this, both cans tested were the same type and size of the can. Each had a piece of foam on top of it to block heat from escaping the thermistor hole. They each had a thermistor inserted into the top at an angle roughly in the middle of the water. They each had the same volume of water (measured by height in an insulated mixing cup). The temperature in the room was also constant between the two tests.

The only difference between the two tests was adding fins to the soda can using a thermal paste and a 12V computer fan blowing on the can. The fins will increase the surface area of the can, allowing it to dissipate heat over more surfaces. The computer fan will keep cool air running over the fan, forcing convection. An image of this setup can be seen below.



The control set up looked the same without the additions of fins and with the fan turned off.

Computations:

During each trial, temperature readings were taken once a minute for 10 minutes. The reason for this is so that the thermistor can accurately describe the rate of heat transfer. This sampling rate will do fine since the temperature should only change by a fraction of a degree per minute.

Each temperature reading averages 10 readings taken over the 59th second and reported on the 60th. This is done to smooth the data produced to gain the most accuracy out of the sensor which can be slightly temperamental.

The heat transfer coefficient is calculated and reported every minute by first calculating the heat transfer rate (q) and then using it to calculate the coefficient of heat transfer (K). The calculation in code can be seen below, where 4.187 is the specific heat capacity of water in kJ/kg K, 298.7 is the room temperature, and all temperatures are recorded in kelvin.

```
q = 4.187*(prev_temp - temp_k)
avg_t = (prev_temp + temp_k)/2
k = q/(avg_t - 298.7)
print("q = " + str(q) + " k = " + str(k))
```

Tests:

A test was done on each setup. Temperature data was collected and used to calculate the heat transfer coefficient (k) in Watts/(m² * K). The average heat transfer coefficients are reported below.

Time(min)	Control:		Test:	
	Temp (K)	K Coeff	Temp(K)	K Coeff
0	340.7526		339.4697	
1	340.2841	0.046902	338.1828	0.134285
2	339.7129	0.057919	336.7094	0.159218
3	339.254	0.047109	335.3262	0.155194
4	338.7121	0.056322	333.9987	0.154552
5	338.1953	0.054433	332.6688	0.16078
6	337.7148	0.051247	331.4308	0.155422
7	337.2572	0.049406	330.2267	0.156926
8	336.5539	0.077077	329.0863	0.154237
9	335.962	0.065978	327.9796	0.155335
10	335.4956	0.052738	326.8613	0.163025

Avg: **0.055913**

Avg: **0.154897**

Results:

The control setup recorded an average heat transfer coefficient of 0.055913 Watts/(m² * K), while the experimental setup recorded an average heat transfer coefficient of 0.154897 Watts/(m² * K). **This represents a 177% increase in the heat transfer coefficient**, significantly exceeding the goals of this project. The only controlled variable that was slightly different was the starting temperature of the water, although this likely had little effect due to the coefficient readings remaining within the same range as the temperature goes down. While there might be some error in the thermistor readings, it has been accurate over multiple tests. It's calibrated in code and uses smoothed averages of data. This leaves almost, if not entirely, all of the heat transfer gain as a function of the increased surface area and forced convection. Based on the heat transfer improvements made by classmates with only increased surface area, it's likely that most of this gained heat transfer would be a consequence of the forced convection created by the fan.

Code Appendix:

```
from machine import Pin
from machine import PWM
from machine import I2C
import utime
import math

import ads1115 as ADS
from analog_in import AnalogIn

# Create I2C object
i2c = I2C(0, scl=Pin(21), sda=Pin(20))

# Create the ADC object
adc = ADS.ADS1115(i2c)
adc.gain = 1

# Create the input channel
voltage_read = AnalogIn(adc, ADS.P0, ADS.P1)

# Coefficient for stienHart equation
b_parameter = 3950 #1/K

#variables for temperature:
v_therm = 0
v_total = 4.97
v_refrence = 0
r_refrence = 9960
current = v_refrence/r_refrence
```

```

r_therm = 0
temp_k = 0
temp_f = 0
# Timing
delay_ms = 100

# Smoothing
max_readings = 10
readings = [0 for i in range(max_readings)]
current_reading = 0
total = 0

def smoothData(volt_mv):
    global total, current_reading

    # Smoothing accumulator
    total += volt_mv
    total -= readings[current_reading]
    readings[current_reading] = volt_mv

    # Move to the next index
    current_reading += 1

    # Reset current_reading index if necessary
    if current_reading >= max_readings:
        current_reading = 0

    # Calculate the average
    avg = round(total / max_readings, 4)
    return avg

def calc_Temp(v_t):
    global v_refrence, current, r_refrence, r_therm, temp_k, temp_c, temp_f, v_total

    v_refrence = v_total - v_t

    current = v_refrence / r_refrence

    r_therm = v_t / current

    #calibration:
    temp_k = 1.0044 * (1 / ((1 / 296.15) + ((1 / 3950.0) * math.log(r_therm / 10590)))) - 0.0896

    temp_c = temp_k - 273.15

    temp_c = temp_c

    temp_f = temp_c * (9 / 5) + 32

loops = 0
prev_temp = 0
mins = 0

```

```

while True:
    temp = 0
    voltage_mv = voltage_read.voltage
    # Get smoothed voltage
    avgVolt = smoothData(voltage_mv)

    # Calculate temperature from voltage
    if(loops == 10):
        loops = 0
        calc_Temp(avgVolt)
        print("Time: "+str(mins) + "mins, temperature (C): "+str(temp_c)+" C, temp (K): " + str(temp_k))
        if(mins >= 1):
            q = 4.187*(prev_temp - temp_k)
            avg_t = (prev_temp + temp_k)/2
            k = q/(avg_t - 298.7)
            print("q = " + str(q) + " k = " + str(k))
            prev_temp = temp_k
            utime.sleep_ms(59000)
            mins+=1
    # Take readings at intervals
    loops+=1
    utime.sleep_ms(delay_ms)

```